TP 1

Python pour la science des données

Jupyter, NumPy, Pandas, visualisation et premier modèle de Machine Learning

Support pédagogique pas à pas, avec code commenté et résultats

Objectifs et plan du TP

Trois parties progressives : chacune s'appuie sur la précédente.

Partie 1

Jupyter et bases de Python

- Lancer Jupyter, comprendre un notebook
- Variables, types, listes, dictionnaires
- Conditions et boucles
- Fonctions et import de bibliothèques

Partie 2

Manipulation de données

- NumPy : tableaux, indexation, opérations, agrégations
- Pandas : CSV, sélection, filtre, tri, groupby
- Valeurs manquantes
- NumPy ou Pandas : que choisir ?

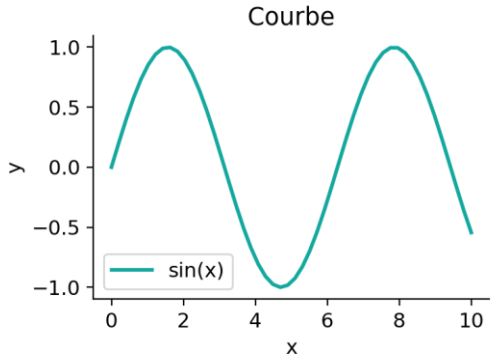
Partie 3

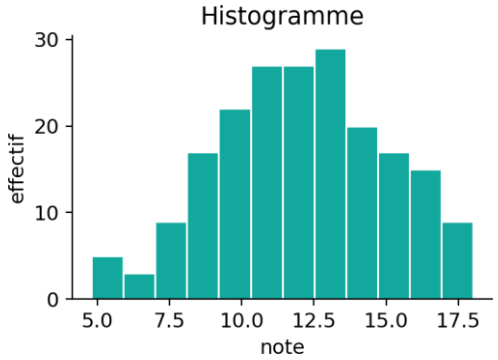
Visualisation et premier ML

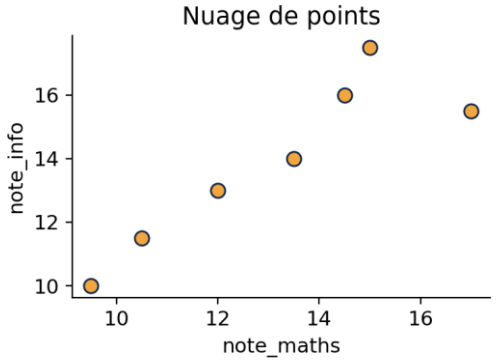
- Matplotlib et Seaborn : courbes, histogrammes, nuages, corrélations
- Scikit-learn : jeu Iris, features et target
- Séparation train/test
- Premier modèle simple

Matplotlib : trois graphiques de base

Matplotlib dessine à partir de données NumPy ou Pandas. On commence toujours par : import matplotlib.pyplot as plt








```
x = np.linspace(0, 10, 50)
plt.plot(x, np.sin(x))
plt.title("Courbe")
plt.show()
```

```
rng = np.random.default_rng(0)
notes = rng.normal(12, 3, 200)
plt.hist(notes, bins=12)
plt.show()
```

```
plt.scatter(df["note_maths"],
            df["note_info"])
plt.xlabel("note_maths")
plt.show()
```

random.default_rng(0): Cette ligne crée un générateur de nombres aléatoires NumPy (rng) initialisé avec une graine (seed) de 0 pour garantir la reproductibilité des résultats

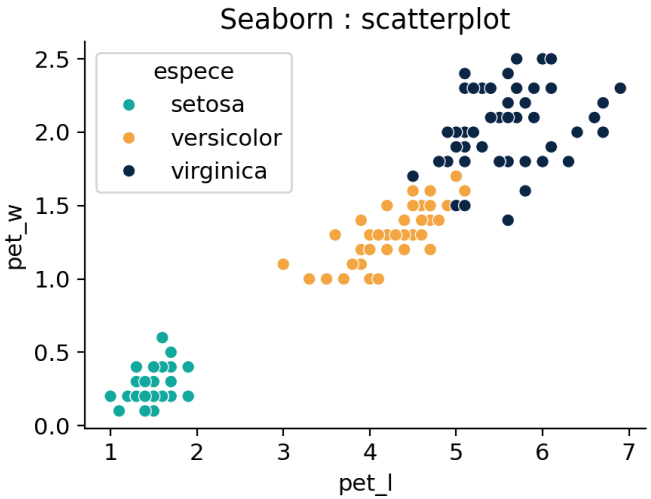
otes = rng.normal(12, 3, 200) : Cette ligne génère un tableau de 200 valeurs aléatoires suivant une distribution normale (gaussienne), avec une moyenne de 12, un écart-type de 3, et stocke le tout dans la variable

Seaborn : des graphiques statistiques en une ligne

Seaborn s'appuie sur Matplotlib et sur Pandas : on donne le DataFrame et

```python
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
X = iris.data
X.columns = ["sep_l", "sep_w", "pet_l", "pet_w"]
d = X.assign(espece=iris.target_names[iris.target])
sns.scatterplot(data=d, x="pet_l", y="pet_w",
                hue="espece")
```



Pourquoi Seaborn ?

- data= : le DataFrame ; x= et y= : les colonnes
- hue= colore automatiquement selon une colonne
- Légende et axes sont ajoutés sans effort
- Iris est détaillé à la diapo « Scikit-learn »

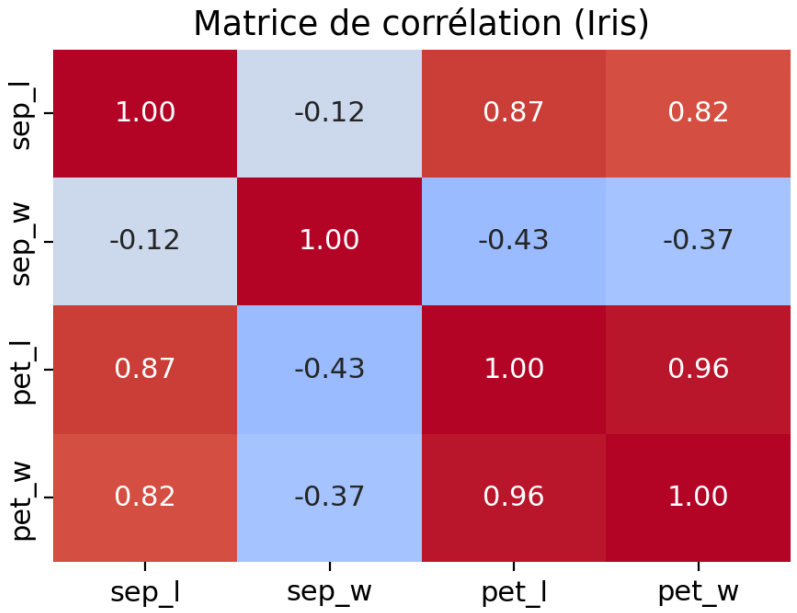
d = X.assign(espece=iris.target_names[iris.target]) : Cette ligne crée un nouveau DataFrame d en y ajoutant une colonne nommée "espece", qui contient le nom textuel de l'espèce de chaque fleur (basé sur la cible numérique et les noms fournis par le jeu de données)

sns.scatterplot(data=d, x="pet_l", y="pet_w", hue="espece") : Cette ligne trace un nuage de points avec Seaborn en plaçant la longueur du pétale ("pet_l") sur l'axe horizontal, la largeur du pétale ("pet_w") sur l'axe vertical, et en colorant les points (hue="espece") différemment selon l'espèce de la fleur.

Matrice de corrélation

La corrélation mesure la relation linéaire entre deux variables : de -1 (inverse) à +1 (même sens), 0 si aucune.

```python
# X : les 4 mesures d'Iris (voir plus loin)
sns.heatmap(X.corr(),
            annot=True,
            cmap="coolwarm")
plt.show()
```



Lire la matrice

- Diagonale = 1 : une variable avec elle-même
- pet_l et pet_w : 0.96, très corrélées
- sep_w et pet_l : -0.43, corrélation négative modérée
- Corrélation n'est pas causalité
- **annot=True** : Ce paramètre indique qu'il faut afficher les valeurs numériques (les coefficients de corrélation) directement à l'intérieur de chaque case de la carte thermique.
- **cmap="coolwarm")** : Ce paramètre définit la palette de couleurs utilisée pour le graphique. La palette "coolwarm" applique des nuances allant du bleu (valeurs négatives) au rouge (valeurs positives), en passant par des tons neutres.

Scikit-learn : le jeu de données Iris

Iris : 150 fleurs de 3 espèces, décrites par 4 mesures. C'est le « Hello World » du Machine Learning.

```python
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
X = iris.data.set_axis(
    ["sep_l", "sep_w", "pet_l", "pet_w"], axis=1)
y = iris.target
print(X.shape, y.shape)
print(X.head(3))
print(iris.target_names.tolist())
print(y.value_counts().to_dict())
```

Features et target

sep_l	sep_w	pet_l	pet_w	y
5.1	3.5	1.4	0.2	0

X = 4 premières colonnes ; y = dernière colonne (0 = setosa)

Résultat

```
(150, 4) (150,)
   sep_l  sep_w  pet_l  pet_w
0    5.1    3.5    1.4    0.2
1    4.9    3.0    1.4    0.2
2    4.7    3.2    1.3    0.2
['setosa', 'versicolor', 'virginica']
{0: 50, 1: 50, 2: 50}
```

À retenir

- X (features) : les 4 mesures, entrées du modèle
- y (target) : l'espèce codée 0, 1 ou 2, à prédire
- 50 fleurs par classe : jeu équilibré

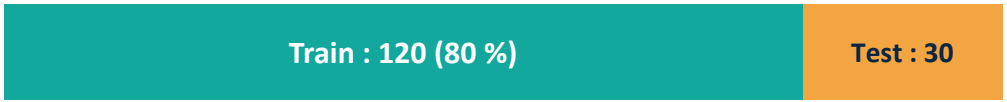
Séparer train et test

On entraîne le modèle sur une partie des données et on l'évalue sur une partie qu'il n'a jamais vue.

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print(X_train.shape, X_test.shape)
```

Les 150 fleurs



- test_size=0.2 : Réserve 20 % des données pour le test et 80 % pour l'entraînement.
- random_state=42 : Fixe une graine aléatoire pour garantir que la division soit reproductible à chaque exécution.
- stratify=y : Assure que la proportion des différentes classes de la cible y soit conservée de manière identique dans les ensembles d'entraînement et de test.

Résultat

```
(120, 4) (30, 4)
```

À retenir

- test_size=0.2 : 20 % pour le test (30 fleurs sur 150)
- random_state=42 : même découpage à chaque exécution
- stratify=y : mêmes proportions d'espèces dans chaque partie

Un premier modèle simple : k plus proches voisins

Le modèle prédit l'espèce d'une fleur d'après ses 3 voisines les plus proches dans le jeu d'entraînement.

Python

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(accuracy_score(y_test, pred))
print(pred[:6], y_test.values[:6])
```

Résultat

```
1.0
[0 2 1 1 0 1] [0 2 1 1 0 1]
```

Attention à l'interprétation

Avec seulement 30 fleurs de test, 1.0 ne prouve pas que le modèle est parfait : le jeu Iris est simple et petit. Un autre découpage peut donner un score un peu différent.

À retenir

- fit : apprentissage sur le train
- predict : prédictions sur le test
- accuracy : part de bonnes prédictions (1.0 = 100 %)

Exercices de la partie 3

À réaliser dans votre notebook, cellule par cellule. Les corrections sont revues à la séance suivante.

1. **Courbes**
   Tracer $y = x^2$ et $y = x^3$ sur la même figure, avec légende, titre et noms d'axes.
2. **Histogramme**
   Tracer l'histogramme de note_maths (après remplacement des NaN) et commenter sa forme.
3. **Nuage de points**
   Représenter note_maths contre note_info et décrire la relation observée.
4. **Corrélation**
   Calculer et afficher la matrice de corrélation des colonnes numériques d'etudiants.csv.
5. **Iris : exploration**
   Charger Iris, afficher la forme de X et la répartition des classes.
6. **Premier modèle**
   Séparer 70/30 (random_state=0), entraîner un KNN à k=3, puis comparer l'accuracy pour k = 1, 3, 5, 7.

Rappel : testez toujours le modèle sur des données non utilisées pour l'entraîner.

Erreurs fréquentes et solutions

Lire le message d'erreur en entier, en commençant par sa dernière ligne : il indique presque toujours la cause.

Erreur	Cause probable	Solution
IndentationError	Bloc mal décalé après if, for, def	4 espaces, toujours les mêmes
NameError	Variable ou import jamais exécuté	Exécuter la cellule, ou Kernel > Restart & Run All
FileNotFoundError	CSV introuvable	Le placer à côté du notebook ; vérifier avec os.getcwd()
KeyError	Nom de colonne incorrect	Comparer avec df.columns (majuscules, accents)
ValueError (shape)	Tableaux de formes incompatibles	Afficher .shape et ajuster avec reshape
Résultat étrange avec and / or	Mauvais opérateur sur Pandas	Utiliser & et |, avec des parenthèses

SYNTHÈSE

Aide-mémoire du TP 1

Les gestes à retenir pour chaque partie.

Python

```
x = 5 ; texte = "a"

liste = [1, 2] ; d = {"k": 1}

if / elif / else

for i in range(n):

def f(x): return x
```

NumPy / Pandas

```
np.array, arange, reshape

a[:, 1] ; a[a > 3] ;
a.sum(axis=0)

pd.read_csv("f.csv")

df[df["c"] > 5] ; df.loc ;
df.iloc

df.groupby("c").mean()  ;  fillna
```

Visualisation et ML

```
plt.plot / hist / scatter

sns.heatmap(df.corr(),
annot=True)

load_iris(as_frame=True)

train_test_split(X, y,
test_size=0.2)

model.fit ; model.predict
```